

C语言与数据结构及算法经典面试题目录

C语言基础

1. 问题: `int main()` 和 `void main()` 的区别是什么?

答案解析: `int main()` 是C语言标准定义的主函数形式, 返回值表示程序执行状态 (0表示成功, 非0表示错误)。 `void main()` 非标准, 可能导致未定义行为, 部分编译器接受但不推荐。

示例代码:

```
int main() {  
    return 0; // 表示程序正常退出  
}
```

2. 问题: `const` 关键字的作用是什么?

答案解析: `const` 用于定义常量, 防止变量被修改。修饰指针时, 需区分 `const int *p` (指针指向的数据不可改) 和 `int *const p` (指针本身不可改)。

示例代码:

```
const int a = 10; // a不可修改  
int b = 20;  
const int *p = &b; // p指向的数据不可改  
// *p = 30; // 错误  
p = &a; // 正确
```

3. 问题: `static` 关键字在C语言中的作用?

答案解析: `static` 修饰局部变量时, 使其生命周期延长至程序结束, 存储在静态存储区; 修饰全局变量或函数时, 限制其作用域为当前文件。

示例代码:

```
void func() {  
    static int count = 0; // 保留值  
    count++;  
    printf("%d\n", count);  
}
```

4. 问题: `volatile` 关键字的作用?

答案解析: `volatile` 告诉编译器变量可能被外部修改, 防止优化 (如寄存器缓存)。常用于多线程或硬件寄存器访问。

示例代码:

```
volatile int flag = 0; // 防止编译器优化
```

5. 问题: `sizeof` 和 `strlen` 的区别?

答案解析: `sizeof` 是操作符, 返回变量或类型的字节大小; `strlen` 是函数, 返回字符串长度 (不含 `\0`) 。

示例代码:

```
char str[] = "hello";  
printf("%zu\n", sizeof(str)); // 6 (含\0)  
printf("%zu\n", strlen(str)); // 5
```

指针与内存管理

6. 问题: 什么是野指针? 如何避免?

答案解析: 野指针是指向非法内存的指针, 可能导致程序崩溃。避免方法: 初始化指针为 `NULL`, 使用后置空, 检查指针有效性。

示例代码:

```
int *p = NULL; // 初始化为NULL  
if (p != NULL) {  
    *p = 10; // 安全访问  
}
```

7. 问题: 指针和引用的区别?

答案解析: 指针是存储地址的变量, 可变且可为空; 引用 (C++特性, C无引用) 是变量别名, 不可变且不可为空。C中常用指针模拟引用行为。

示例代码:

```
void swap(int *a, int *b) {  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

8. 问题: 解释内存泄漏及其检测方法。

答案解析: 内存泄漏指动态分配的内存未释放。检测方法: 使用工具如Valgrind, 手动检查 `malloc/free` 配对。

示例代码:

```
int *p = malloc(sizeof(int));  
if (p) {  
    *p = 10;  
    free(p); // 防止泄漏  
    p = NULL;  
}
```

9. 问题: `malloc` 和 `calloc` 的区别?

答案解析: `malloc` 分配指定字节内存, 未初始化; `calloc` 分配并初始化为0。

示例代码:

```
int *p1 = malloc(10 * sizeof(int)); // 未初始化
int *p2 = calloc(10, sizeof(int)); // 初始化为0
```

10. 问题: 什么是段错误 (Segmentation Fault) ?

答案解析: 段错误通常由访问非法内存引起, 如解引用NULL指针、越界访问数组等。

示例代码:

```
int *p = NULL;
// *p = 10; // 段错误
```

数据结构

链表

11. 问题: 实现单链表的逆序。

答案解析: 遍历链表, 逐个改变指针方向, 需三个指针 (`prev`, `curr`, `next`) 防止断链。时间复杂度 $O(n)$, 空间复杂度 $O(1)$ 。

示例代码:

```
struct ListNode {
    int val;
    struct ListNode *next;
};

struct ListNode* reverseList(struct ListNode* head) {
    struct ListNode *prev = NULL, *curr = head, *next;
    while (curr) {
        next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
```

12. 问题: 检测链表中是否有环。

答案解析: 使用快慢指针 (Floyd算法), 快指针每次走两步, 慢指针走一步, 若相遇则有环。时间复杂度 $O(n)$, 空间复杂度 $O(1)$ 。

示例代码:

```
bool hasCycle(struct ListNode *head) {
    struct ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}
```

13. **问题：**合并两个有序链表。

答案解析：递归或迭代比较节点值，构造新链表。时间复杂度 $O(n+m)$ ，空间复杂度 $O(1)$ （迭代）或 $O(n+m)$ （递归）。

示例代码：

```
struct ListNode* mergeTwoLists(struct ListNode* l1, struct ListNode* l2) {
    struct ListNode dummy, *tail = &dummy;
    dummy.next = NULL;
    while (l1 && l2) {
        if (l1->val < l2->val) {
            tail->next = l1;
            l1 = l1->next;
        } else {
            tail->next = l2;
            l2 = l2->next;
        }
        tail = tail->next;
    }
    tail->next = l1 ? l1 : l2;
    return dummy.next;
}
```

14. **问题：**删除链表倒数第N个节点。

答案解析：使用双指针，快指针先走N步，慢指针跟进，找到倒数第N个节点前驱，删除目标节点。时间复杂度 $O(n)$ ，空间复杂度 $O(1)$ 。

示例代码：

```
struct ListNode* removeNthFromEnd(struct ListNode* head, int n) {
    struct ListNode dummy = {0, head}, *fast = &dummy, *slow = &dummy;
    for (int i = 0; i <= n; i++) fast = fast->next;
    while (fast) {
        fast = fast->next;
        slow = slow->next;
    }
    slow->next = slow->next->next;
    return dummy.next;
}
```

15. **问题：**查找链表中间节点。

答案解析：快慢指针法，慢指针走一步，快指针走两步，快指针到末尾时慢指针指向中间。时间复杂度 $O(n)$ ，空间复杂度 $O(1)$ 。

示例代码：

```
struct ListNode* middleNode(struct ListNode* head) {
    struct ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}
```

栈与队列

16. **问题：**用两个栈实现队列。

答案解析：一个栈用于入队，另一个栈用于出队，需将入队栈元素倒序压入出队栈。入队 $O(1)$ ，出队均摊 $O(1)$ 。

示例代码：

```
typedef struct {
    int *stack1, *stack2;
    int top1, top2, capacity;
} MyQueue;

MyQueue* myQueueCreate(int capacity) {
    MyQueue *q = malloc(sizeof(MyQueue));
    q->stack1 = malloc(capacity * sizeof(int));
    q->stack2 = malloc(capacity * sizeof(int));
    q->top1 = q->top2 = -1;
    q->capacity = capacity;
    return q;
}

void myQueuePush(MyQueue* obj, int x) {
    obj->stack1[++obj->top1] = x;
}

int myQueuePop(MyQueue* obj) {
    if (obj->top2 == -1) {
        while (obj->top1 >= 0)
            obj->stack2[++obj->top2] = obj->stack1[obj->top1--];
    }
    return obj->stack2[obj->top2--];
}
```

17. **问题：**实现一个栈，支持获取最小元素（MinStack）。

答案解析：用两个栈，一个存数据，一个存当前最小值。每次压入时，更新最小值栈。时间复杂度

$O(1)$ 。

示例代码：

```
typedef struct {
    int *data, *min;
    int top, capacity;
} MinStack;
MinStack* minStackCreate(int capacity) {
    MinStack *s = malloc(sizeof(MinStack));
    s->data = malloc(capacity * sizeof(int));
    s->min = malloc(capacity * sizeof(int));
    s->top = -1;
    s->capacity = capacity;
    return s;
}
void minStackPush(MinStack* obj, int x) {
    obj->data[++obj->top] = x;
    obj->min[obj->top] = (obj->top == 0 || x < obj->min[obj->top-1]) ? x :
obj->min[obj->top-1];
}
int minStackGetMin(MinStack* obj) {
    return obj->min[obj->top];
}
```

18. **问题：**判断括号字符串是否有效。

答案解析：用栈检查括号匹配，遇到左括号入栈，右括号检查栈顶是否匹配。时间复杂度 $O(n)$ ，空间复杂度 $O(n)$ 。

示例代码：

```
bool isValid(char *s) {
    char stack[10000];
    int top = -1;
    for (int i = 0; s[i]; i++) {
        if (s[i] == '(' || s[i] == '[' || s[i] == '{') {
            stack[++top] = s[i];
        } else {
            if (top == -1) return false;
            if (s[i] == ')' && stack[top] != '(') return false;
            if (s[i] == ']' && stack[top] != '[') return false;
            if (s[i] == '}' && stack[top] != '{') return false;
            top--;
        }
    }
    return top == -1;
}
```

树

19. **问题：**实现二叉树的前序遍历。

答案解析：前序遍历（根-左-右）可用递归或迭代（栈）。时间复杂度 $O(n)$ ，空间复杂度 $O(h)$ ， h 为树高。

示例代码：

```
struct TreeNode {
    int val;
    struct TreeNode *left, *right;
};
void preorderTraversal(struct TreeNode* root, int* res, int* returnSize) {
    if (!root) return;
    res[(*returnSize)++] = root->val;
    preorderTraversal(root->left, res, returnSize);
    preorderTraversal(root->right, res, returnSize);
}
```

20. **问题：**判断二叉树是否为二叉搜索树（BST）。

答案解析：中序遍历BST应为递增序列，检查遍历结果是否满足。时间复杂度 $O(n)$ ，空间复杂度 $O(h)$ 。

示例代码：

```
bool isValidBST(struct TreeNode* root) {
    static struct TreeNode *prev = NULL;
    if (!root) return true;
    if (!isValidBST(root->left)) return false;
    if (prev && prev->val >= root->val) return false;
    prev = root;
    return isValidBST(root->right);
}
```

21. **问题：**求二叉树的最大深度。

答案解析：递归计算左右子树深度，取最大值加1。时间复杂度 $O(n)$ ，空间复杂度 $O(h)$ 。

示例代码：

```
int maxDepth(struct TreeNode* root) {
    if (!root) return 0;
    int left = maxDepth(root->left);
    int right = maxDepth(root->right);
    return (left > right ? left : right) + 1;
}
```

22. **问题：**实现二叉树的层序遍历。

答案解析：使用队列，按层遍历节点。时间复杂度 $O(n)$ ，空间复杂度 $O(w)$ ， w 为最大层宽。

示例代码：

```

int** levelOrder(struct TreeNode* root, int* returnSize, int**
returnColumnSizes) {
    if (!root) return NULL;
    int **res = malloc(1000 * sizeof(int*));
    *returnColumnSizes = malloc(1000 * sizeof(int));
    struct TreeNode *queue[1000];
    int front = 0, rear = 0;
    queue[rear++] = root;
    *returnSize = 0;
    while (front < rear) {
        int levelSize = rear - front;
        res[*returnSize] = malloc(levelSize * sizeof(int));
        (*returnColumnSizes)[*returnSize] = levelSize;
        for (int i = 0; i < levelSize; i++) {
            struct TreeNode *node = queue[front++];
            res[*returnSize][i] = node->val;
            if (node->left) queue[rear++] = node->left;
            if (node->right) queue[rear++] = node->right;
        }
        (*returnSize)++;
    }
    return res;
}

```

23. **问题：**判断二叉树是否对称。

答案解析：递归比较左右子树，左子树的左孩子与右子树的右孩子、左子树的右孩子与右子树的左孩子对称。时间复杂度 $O(n)$ 。

示例代码：

```

bool isSymmetric(struct TreeNode* root) {
    if (!root) return true;
    return isMirror(root->left, root->right);
}

bool isMirror(struct TreeNode* left, struct TreeNode* right) {
    if (!left && !right) return true;
    if (!left || !right) return false;
    return left->val == right->val &&
        isMirror(left->left, right->right) &&
        isMirror(left->right, right->left);
}

```

图

24. **问题：**实现图的深度优先搜索（DFS）。

答案解析：用递归或栈遍历图，访问未标记节点并标记。时间复杂度 $O(V+E)$ ，空间复杂度 $O(V)$ 。

示例代码：


```
void dfs(int** graph, int V, int node, bool* visited) {
    visited[node] = true;
    printf("%d ", node);
    for (int i = 0; i < V; i++) {
        if (graph[node][i] && !visited[i]) {
            dfs(graph, V, i, visited);
        }
    }
}
```

25. **问题：**实现图的广度优先搜索（BFS）。

答案解析：用队列遍历图，按层访问节点。时间复杂度 $O(V+E)$ ，空间复杂度 $O(V)$ 。

示例代码：

```
void bfs(int** graph, int V, int start) {
    bool *visited = calloc(V, sizeof(bool));
    int queue[1000], front = 0, rear = 0;
    queue[rear++] = start;
    visited[start] = true;
    while (front < rear) {
        int node = queue[front++];
        printf("%d ", node);
        for (int i = 0; i < V; i++) {
            if (graph[node][i] && !visited[i]) {
                queue[rear++] = i;
                visited[i] = true;
            }
        }
    }
    free(visited);
}
```

算法

排序

26. **问题：**实现冒泡排序。

答案解析：两两比较相邻元素，交换顺序错误的元素。时间复杂度 $O(n^2)$ ，空间复杂度 $O(1)$ 。

示例代码：

```
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n-1; i++)
        for (int j = 0; j < n-i-1; j++)
            if (arr[j] > arr[j+1]) {
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
}
```

```
    }  
}
```

27. **问题：**实现快速排序。

答案解析：选择基准，分区后递归排序。平均时间复杂度 $O(n \log n)$ ，空间复杂度 $O(\log n)$ 。

示例代码：

```
void quickSort(int arr[], int low, int high) {  
    if (low < high) {  
        int pi = partition(arr, low, high);  
        quickSort(arr, low, pi-1);  
        quickSort(arr, pi+1, high);  
    }  
}  
  
int partition(int arr[], int low, int high) {  
    int pivot = arr[high];  
    int i = low - 1;  
    for (int j = low; j < high; j++) {  
        if (arr[j] <= pivot) {  
            i++;  
            int temp = arr[i];  
            arr[i] = arr[j];  
            arr[j] = temp;  
        }  
    }  
    int temp = arr[i+1];  
    arr[i+1] = arr[high];  
    arr[high] = temp;  
    return i+1;  
}
```

28. **问题：**实现归并排序。

答案解析：分治法，将数组分成两半递归排序后合并。时间复杂度 $O(n \log n)$ ，空间复杂度 $O(n)$ 。

示例代码：

```
void mergeSort(int arr[], int l, int r) {  
    if (l < r) {  
        int m = l + (r - l) / 2;  
        mergeSort(arr, l, m);  
        mergeSort(arr, m+1, r);  
        merge(arr, l, m, r);  
    }  
}  
  
void merge(int arr[], int l, int m, int r) {  
    int n1 = m - l + 1, n2 = r - m;  
    int L[n1], R[n2];  
    for (int i = 0; i < n1; i++) L[i] = arr[l+i];  
    for (int i = 0; i < n2; i++) R[i] = arr[m+1+i];  
    int i = 0, j = 0, k = l;
```

```

while (i < n1 && j < n2) {
    if (L[i] <= R[j]) arr[k++] = L[i++];
    else arr[k++] = R[j++];
}
while (i < n1) arr[k++] = L[i++];
while (j < n2) arr[k++] = R[j++];
}

```

29. **问题：**实现堆排序。

答案解析：构建最大堆，依次将堆顶与末尾交换并调整堆。时间复杂度 $O(n \log n)$ ，空间复杂度 $O(1)$ 。

示例代码：

```

void heapSort(int arr[], int n) {
    for (int i = n/2-1; i >= 0; i--)
        heapify(arr, n, i);
    for (int i = n-1; i > 0; i--) {
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;
        heapify(arr, i, 0);
    }
}

void heapify(int arr[], int n, int i) {
    int largest = i, left = 2*i+1, right = 2*i+2;
    if (left < n && arr[left] > arr[largest]) largest = left;
    if (right < n && arr[right] > arr[largest]) largest = right;
    if (largest != i) {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;
        heapify(arr, n, largest);
    }
}

```

30. **问题：**比较各种排序算法的优缺点。

答案解析：

- 冒泡排序：简单，稳定， $O(n^2)$ ，适合小数据。
- 快速排序：平均 $O(n \log n)$ ，不稳定，适合大多数场景。
- 归并排序：稳定， $O(n \log n)$ ，需额外空间，适合链表。
- 堆排序： $O(n \log n)$ ，不稳定，适合优先级队列。

查找

31. **问题：**实现二分查找。

答案解析：在有序数组中查找目标，比较中间元素，递归或迭代缩小范围。时间复杂度 $O(\log n)$ 。

示例代码：

```
int binarySearch(int arr[], int l, int r, int x) {
    while (l <= r) {
        int m = l + (r - l) / 2;
        if (arr[m] == x) return m;
        if (arr[m] < x) l = m + 1;
        else r = m - 1;
    }
    return -1;
}
```

32. **问题：**实现二叉搜索树（BST）的插入操作。

答案解析：递归比较节点值，插入到合适位置。时间复杂度 $O(h)$ ， h 为树高。

示例代码：

```
struct TreeNode* insertBST(struct TreeNode* root, int val) {
    if (!root) {
        struct TreeNode* node = malloc(sizeof(struct TreeNode));
        node->val = val;
        node->left = node->right = NULL;
        return node;
    }
    if (val < root->val)
        root->left = insertBST(root->left, val);
    else
        root->right = insertBST(root->right, val);
    return root;
}
```

递归与动态规划

33. **问题：**实现斐波那契数列（递归与迭代）。

答案解析：递归简单但效率低（ $O(2^n)$ ），迭代或动态规划 $O(n)$ 。

示例代码：

```
// 迭代
int fib(int n) {
    if (n <= 1) return n;
    int a = 0, b = 1, c;
    for (int i = 2; i <= n; i++) {
        c = a + b;
        a = b;
        b = c;
    }
    return b;
}
```

34. **问题：**实现阶乘函数（递归与迭代）。

答案解析：递归直观，迭代避免栈溢出。时间复杂度 $O(n)$ 。

示例代码：

```
// 迭代
unsigned long long factorial(int n) {
    unsigned long long res = 1;
    for (int i = 1; i <= n; i++) res *= i;
    return res;
}
```

35. **问题：**解决背包问题（0-1背包）。

答案解析：动态规划， $dp[i][w]$ 表示前 i 个物品在容量 w 下的最大价值。时间复杂度 $O(n*W)$ 。

示例代码：

```
int knapsack(int W, int wt[], int val[], int n) {
    int dp[n+1][W+1];
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0) dp[i][w] = 0;
            else if (wt[i-1] <= w)
                dp[i][w] = (val[i-1] + dp[i-1][w-wt[i-1]]) > dp[i-1][w] ?
                    val[i-1] + dp[i-1][w-wt[i-1]] : dp[i-1][w];
            else
                dp[i][w] = dp[i-1][w];
        }
    }
    return dp[n][W];
}
```

字符串

36. **问题：**实现字符串反转。

答案解析：双指针从两端交换字符。时间复杂度 $O(n)$ ，空间复杂度 $O(1)$ 。

示例代码：

```
void reverseString(char* s, int sSize) {
    for (int i = 0, j = sSize-1; i < j; i++, j--) {
        char temp = s[i];
        s[i] = s[j];
        s[j] = temp;
    }
}
```

37. **问题：**实现字符串匹配（KMP算法）。

答案解析：KMP通过部分匹配表避免重复比较。时间复杂度 $O(n+m)$ 。

示例代码：

```
void getNext(char* pattern, int next[], int len) {
    next[0] = -1;
    int i = 0, j = -1;
    while (i < len-1) {
        if (j == -1 || pattern[i] == pattern[j]) {
            i++; j++;
            next[i] = j;
        } else {
            j = next[j];
        }
    }
}

int strStr(char* haystack, char* needle) {
    int m = strlen(needle), n = strlen(haystack);
    if (m == 0) return 0;
    int next[m];
    getNext(needle, next, m);
    int i = 0, j = 0;
    while (i < n && j < m) {
        if (j == -1 || haystack[i] == needle[j]) {
            i++; j++;
        } else {
            j = next[j];
        }
    }
    return j == m ? i - m : -1;
}
```

38. **问题：**判断字符串是否为回文。

答案解析：双指针比较首尾字符，忽略非字母数字。时间复杂度O(n)。

示例代码：

```
bool isPalindrome(char *s) {
    int left = 0, right = strlen(s)-1;
    while (left < right) {
        while (left < right && !isalnum(s[left])) left++;
        while (left < right && !isalnum(s[right])) right--;
        if (tolower(s[left++]) != tolower(s[right--])) return false;
    }
    return true;
}
```

其他算法

39. **问题：**实现两数之和（给定数组和目标值，找两个数的下标）。

答案解析：用哈希表存储已遍历元素，查找差值。时间复杂度O(n)。

示例代码:

```

int* twoSum(int* nums, int numsSize, int target, int* returnSize) {
    int *hash = calloc(100000, sizeof(int));
    int *res = malloc(2 * sizeof(int));
    *returnSize = 2;
    for (int i = 0; i < numsSize; i++) {
        int diff = target - nums[i];
        if (hash[diff+50000]) {
            res[0] = hash[diff+50000] - 1;
            res[1] = i;
            free(hash);
            return res;
        }
        hash[nums[i]+50000] = i + 1;
    }
    free(hash);
    return NULL;
}

```

40. **问题:** 实现LRU缓存 (最近最少使用) 。

答案解析: 用哈希表和双向链表实现, 哈希表快速定位节点, 双向链表维护访问顺序。时间复杂度 $O(1)$ 。

示例代码:

```

typedef struct Node {
    int key, val;
    struct Node *prev, *next;
} Node;
typedef struct {
    int capacity;
    Node *head, *tail;
    Node **hash;
} LRUCache;
LRUCache* LRUCacheCreate(int capacity) {
    LRUCache *cache = malloc(sizeof(LRUCache));
    cache->capacity = capacity;
    cache->hash = calloc(10000, sizeof(Node*));
    cache->head = malloc(sizeof(Node));
    cache->tail = malloc(sizeof(Node));
    cache->head->next = cache->tail;
    cache->tail->prev = cache->head;
    return cache;
}
int LRUCacheGet(LRUCache* obj, int key) {
    Node *node = obj->hash[key];
    if (!node) return -1;
    node->prev->next = node->next;
    node->next->prev = node->prev;
    node->next = obj->head->next;
}

```

```

    node->prev = obj->head;
    obj->head->next->prev = node;
    obj->head->next = node;
    return node->val;
}
void LRUCachePut(LRUCache* obj, int key, int value) {
    if (obj->hash[key]) {
        LRUCacheGet(obj, key);
        obj->hash[key]->val = value;
        return;
    }
    Node *node = malloc(sizeof(Node));
    node->key = key;
    node->val = value;
    obj->hash[key] = node;
    node->next = obj->head->next;
    node->prev = obj->head;
    obj->head->next->prev = node;
    obj->head->next = node;
    if (--obj->capacity < 0) {
        Node *last = obj->tail->prev;
        last->prev->next = obj->tail;
        obj->tail->prev = last->prev;
        obj->hash[last->key] = NULL;
        free(last);
        obj->capacity++;
    }
}

```

41. 问题：实现atoi（字符串转整数）。

答案解析：处理前导空格、符号、数字，注意溢出。时间复杂度O(n)。

示例代码：

```

int myAtoi(char *str) {
    int sign = 1, res = 0, i = 0;
    while (str[i] == ' ') i++;
    if (str[i] == '+' || str[i] == '-') {
        sign = str[i] == '+' ? 1 : -1;
        i++;
    }
    while (isdigit(str[i])) {
        if (res > INT_MAX / 10 || (res == INT_MAX / 10 && str[i] > '7'))
            return sign == 1 ? INT_MAX : INT_MIN;
        res = res * 10 + (str[i] - '0');
        i++;
    }
    return sign * res;
}

```


42. **问题：**实现最大子序和。

答案解析：Kadane算法，动态规划维护当前最大子序和。时间复杂度 $O(n)$ 。

示例代码：

```
int maxSubArray(int* nums, int numsSize) {
    int maxSum = nums[0], currSum = nums[0];
    for (int i = 1; i < numsSize; i++) {
        currSum = (currSum > 0 ? currSum : 0) + nums[i];
        maxSum = currSum > maxSum ? currSum : maxSum;
    }
    return maxSum;
}
```

43. **问题：**实现三数之和（找到所有和为0的唯一三元组）。

答案解析：排序后固定一个数，用双指针找另外两个数。时间复杂度 $O(n^2)$ 。

示例代码：

```
int** threeSum(int* nums, int numsSize, int* returnSize, int**
returnColumnSizes) {
    qsort(nums, numsSize, sizeof(int), cmp);
    int **res = malloc(10000 * sizeof(int*));
    *returnColumnSizes = malloc(10000 * sizeof(int));
    *returnSize = 0;
    for (int i = 0; i < numsSize-2; i++) {
        if (i > 0 && nums[i] == nums[i-1]) continue;
        int left = i+1, right = numsSize-1;
        while (left < right) {
            int sum = nums[i] + nums[left] + nums[right];
            if (sum == 0) {
                res[*returnSize] = malloc(3 * sizeof(int));
                res[*returnSize][0] = nums[i];
                res[*returnSize][1] = nums[left];
                res[*returnSize][2] = nums[right];
                (*returnColumnSizes)[*returnSize] = 3;
                (*returnSize)++;
                while (left < right && nums[left] == nums[left+1]) left++;
                while (left < right && nums[right] == nums[right-1]) right--;
            }
            else if (sum < 0) left++;
            else right--;
        }
    }
    return res;
}

int cmp(const void* a, const void* b) {
    return *(int*)a - *(int*)b;
}
```

44. **问题：**实现矩阵旋转（顺时针90度）。

答案解析：先沿主对角线翻转，再左右翻转。时间复杂度 $O(n^2)$ ，空间复杂度 $O(1)$ 。

示例代码：

```
void rotate(int** matrix, int matrixSize, int* matrixColSize) {
    for (int i = 0; i < matrixSize; i++)
        for (int j = i; j < matrixSize; j++) {
            int temp = matrix[i][j];
            matrix[i][j] = matrix[j][i];
            matrix[j][i] = temp;
        }
    for (int i = 0; i < matrixSize; i++)
        for (int j = 0; j < matrixSize/2; j++) {
            int temp = matrix[i][j];
            matrix[i][j] = matrix[i][matrixSize-1-j];
            matrix[i][matrixSize-1-j] = temp;
        }
}
```

45. **问题：**实现最长公共前缀。

答案解析：逐字符比较所有字符串的相同位置。时间复杂度 $O(m*n)$ ， m 为最短字符串长度， n 为字符串数。

示例代码：

```
char* longestCommonPrefix(char** strs, int strSize) {
    if (strSize == 0) return "";
    for (int i = 0; ; i++) {
        char c = strs[0][i];
        if (!c) return strs[0];
        for (int j = 1; j < strSize; j++)
            if (strs[j][i] != c || !strs[j][i])
                return strdup(strs[0], i);
    }
}
```

46. **问题：**实现岛屿数量（网格中）。

答案解析：DFS或BFS遍历网格，遇到1标记并递归/队列扩展。时间复杂度 $O(m*n)$ 。

示例代码：

```
void dfsIsland(char** grid, int m, int n, int i, int j) {
    if (i < 0 || i >= m || j < 0 || j >= n || grid[i][j] != '1') return;
    grid[i][j] = '0';
    dfsIsland(grid, m, n, i+1, j);
    dfsIsland(grid, m, n, i-1, j);
    dfsIsland(grid, m, n, i, j+1);
    dfsIsland(grid, m, n, i, j-1);
}
```

```
int numIslands(char** grid, int gridSize, int* gridColSize) {
    int count = 0;
    for (int i = 0; i < gridSize; i++)
        for (int j = 0; j < *gridColSize; j++)
            if (grid[i][j] == '1') {
                dfsIsland(grid, gridSize, *gridColSize, i, j);
                count++;
            }
    return count;
}
```

47. **问题：**实现滑动窗口最大值。

答案解析：用双端队列维护窗口内最大值的下标，队首为最大值。时间复杂度 $O(n)$ 。

示例代码：

```
int* maxSlidingWindow(int* nums, int numsSize, int k, int* returnSize) {
    int *res = malloc(numsSize * sizeof(int));
    int deque[10000], front = 0, rear = 0;
    *returnSize = 0;
    for (int i = 0; i < numsSize; i++) {
        while (front < rear && nums[deque[rear-1]] <= nums[i]) rear--;
        deque[rear++] = i;
        if (deque[front] <= i-k) front++;
        if (i >= k-1) res[(*returnSize)++] = nums[deque[front]];
    }
    return res;
}
```

48. **问题：**实现最小覆盖子串。

答案解析：滑动窗口，维护字符计数，缩小窗口找到最小覆盖。时间复杂度 $O(n)$ 。

示例代码：

```
char* minWindow(char* s, char* t) {
    int need[128] = {0}, have[128] = {0}, count = 0, minLen = INT_MAX, start = 0;
    for (int i = 0; t[i]; i++) need[t[i]]++;
    for (int i = 0; t[i]; i++) if (need[t[i]]) count++;
    int left = 0, matched = 0;
    for (int right = 0; s[right]; right++) {
        if (need[s[right]] && ++have[s[right]] == need[s[right]]) matched++;
        while (matched == count) {
            if (right - left + 1 < minLen) {
                minLen = right - left + 1;
                start = left;
            }
            if (need[s[left]] && --have[s[left]] < need[s[left]]) matched--;
            left++;
        }
    }
}
```

```

    return minLen == INT_MAX ? "" : strdup(s + start, minLen);
}

```

49. **问题：**实现N皇后问题。

答案解析：回溯法，逐行放置皇后，检查冲突。时间复杂度 $O(n!)$ 。

示例代码：

```

bool isSafe(int** board, int row, int col, int n) {
    for (int i = 0; i < row; i++)
        if (board[i][col]) return false;
    for (int i = row, j = col; i >= 0 && j >= 0; i--, j--)
        if (board[i][j]) return false;
    for (int i = row, j = col; i >= 0 && j < n; i--, j++)
        if (board[i][j]) return false;
    return true;
}

void solveNQueens(int** board, int row, int n, int* returnSize, int*** res)
{
    if (row == n) {
        res[*returnSize] = malloc(n * sizeof(int*));
        for (int i = 0; i < n; i++) {
            res[*returnSize][i] = malloc(n * sizeof(int));
            memcpy(res[*returnSize][i], board[i], n * sizeof(int));
        }
        (*returnSize)++;
        return;
    }
    for (int col = 0; col < n; col++) {
        if (isSafe(board, row, col, n)) {
            board[row][col] = 1;
            solveNQueens(board, row+1, n, returnSize, res);
            board[row][col] = 0;
        }
    }
}

```

50. **问题：**实现Dijkstra算法（单源最短路径）。

答案解析：用优先队列（最小堆）维护距离，贪心选择最小距离节点。时间复杂度 $O((V+E)\log V)$ 。

示例代码：

```

typedef struct {
    int node, dist;
} Pair;

void dijkstra(int** graph, int V, int src, int dist[]) {
    int *visited = calloc(V, sizeof(int));
    Pair *pq = malloc(V * sizeof(Pair));
    int size = 0;
    for (int i = 0; i < V; i++) dist[i] = INT_MAX;
    dist[src] = 0;
}

```

```
    pq[size++] = (Pair){src, 0};
    while (size) {
        int u = pq[0].node;
        int d = pq[0].dist;
        for (int i = 0; i < size-1; i++) pq[i] = pq[i+1];
        size--;
        if (visited[u]) continue;
        visited[u] = 1;
        for (int v = 0; v < V; v++) {
            if (graph[u][v] && dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
                pq[size++] = (Pair){v, dist[v]};
                for (int i = size-1; i > 0 && pq[i].dist < pq[i-1].dist; i-
- )
                    pq[i] = pq[i-1];
            }
        }
    }
    free(visited);
    free(pq);
}
```
